

Laboratório 01 - Características de Repositórios Populares

Lucas Flor Vilela, Marina Ferreira Sansão Cabalzar

6 de março de 2026

Resumo

O desenvolvimento de software de código aberto (open-source) tornou-se um pilar fundamental na engenharia de software moderna. Plataformas como o GitHub hospedam milhões de projetos, mas apenas uma pequena parcela atinge um alto nível de popularidade e engajamento. Este artigo apresenta um estudo empírico com o objetivo de caracterizar os 1.000 repositórios mais populares do GitHub. Para tanto, utilizou-se a API GraphQL do GitHub para extrair e analisar métricas fundamentais, incluindo a idade do repositório, volume de contribuições externas (Pull Requests), frequência de lançamentos (Releases) e atualizações, linguagens de programação predominantes e a taxa de resolução de problemas (Issues). A metodologia baseia-se na coleta automatizada de dados via script em Python e na sumarização quantitativa através de valores medianos. Espera-se que as análises forneçam insights valiosos sobre o nível de maturidade, as práticas de manutenção e o engajamento da comunidade por trás dos sistemas open-source de maior sucesso na atualidade.

1 Introdução

1.1 Contextualização

O desenvolvimento de software open-source (código aberto) transformou a maneira como sistemas são construídos e mantidos globalmente. Plataformas de hospedagem de código, como o GitHub, abrigam milhões de repositórios, mas apenas uma pequena fração atinge um alto nível de popularidade (medido pelo número de "estrelas"). Entender o que diferencia esses repositórios altamente populares dos demais é fundamental para compreender as melhores práticas de engenharia de software na comunidade open-source.

1.2 Problema Foco do Experimento

O experimento busca entender quais são as características estruturais e de desenvolvimento que marcam os sistemas open-source mais populares. O problema central consiste em investigar se o sucesso de um repositório está atrelado à sua idade, frequência de atualizações, volume de contribuições externas (Pull Requests), constância de novas versões (Releases), linguagens de programação escolhidas e eficiência na resolução de problemas (Issues).

1.3 Questões Problema

Para guiar este estudo, foram definidas as seguintes Questões de Pesquisa (RQs):

1. Sistemas populares são maduros/antigos?
2. Sistemas populares recebem muita contribuição externa?
3. Sistemas populares lançam releases com frequência?
4. Sistemas populares são atualizados com frequência?
5. Sistemas populares são escritos nas linguagens mais populares?
6. Sistemas populares possuem um alto percentual de issues fechadas?

1.4 Hipóteses

De forma informal, estabelecemos as seguintes expectativas antes da análise dos dados:

- Espera-se que os sistemas mais populares sejam relativamente maduros (antigos), pois acumular milhares de estrelas leva tempo.
- Acredita-se que haverá um alto número de contribuições externas (Pull Requests aceitas) e uma alta taxa de resolução de problemas (alto percentual de Issues fechadas), indicando comunidades ativas.
- Espera-se que as linguagens de programação predominantes reflitam o mercado atual (como JavaScript, Python e TypeScript).
- Supõe-se que esses sistemas recebam atualizações e lancem releases de forma contínua e frequente.

1.5 Objetivo

- Principal: Analisar as características de desenvolvimento e manutenção dos 1.000 repositórios mais populares do GitHub.
- Coletar dados dos repositórios via API GraphQL do GitHub através de um script automatizado. Processar e analisar as métricas coletadas.

2 Metodologia

2.1 Passo a passo do experimento

- Elaboração de uma query em GraphQL capaz de buscar repositórios ordenados por número de estrelas (ordem decrescente) e extrair seus respectivos nós de dados (data de criação, atualização, linguagem primária, contagem de PRs merged, releases e issues).
- Desenvolvimento de um script em Python para consumir a API do GitHub de forma automatizada, implementando paginação para coletar lotes de dados até atingir o total de 1.000 repositórios.
- Conversão dos dados coletados do formato JSON para o formato CSV.
- Análise quantitativa dos dados no CSV para responder às Questões de Pesquisa elaboradas.

2.2 Decisões

Optou-se por utilizar paginação com lotes pequenos (ex: 10 a 20 repositórios por requisição) e mecanismos de retry (tentativa) no script Python para evitar o esgotamento do limite de requisições e contornar erros de sobrecarga (Bad Gateway 502) nos servidores do GitHub. Além disso, seguindo as diretrizes do laboratório, decidiu-se não utilizar bibliotecas de terceiros específicas para o GitHub (como PyGithub), realizando o consumo direto da API via requisições HTTP padrão.

2.3 Materiais utilizados

- Linguagem de programação: Python 3.
- Bibliotecas: requests (para comunicação HTTP) e json / csv (para manipulação de dados).
- Ambiente de Desenvolvimento: Visual Studio Code (VS Code).
- Fonte de Dados: API GraphQL do GitHub v4.

2.4 Métodos utilizados

O método baseia-se em uma abordagem quantitativa exploratória. Para evitar distorções causadas por valores extremos (outliers), a sumarização dos dados numéricos para responder às RQs 01, 02, 03, 04 e 06 será feita utilizando a mediana. Para a RQ 05 (linguagens de programação), será utilizado o método de contagem de frequência (moda).

2.5 Métricas e suas unidades

- Idade do repositório (RQ 01): Calculada em anos/dias a partir do campo `createdAt` até a data da coleta.
- Contribuição externa (RQ 02): Unidade inteira, representada pela contagem de Pull Requests com status `MERGED`.
- Frequência de releases (RQ 03): Unidade inteira, representada pelo número total de releases.
- Frequência de atualização (RQ 04): Calculada em dias a partir da data atual subtraída do campo `updatedAt`.
- Linguagem primária (RQ 05): Categórica (String), obtida do campo `primaryLanguage`.
- Engajamento de issues (RQ 06): Percentual (%), calculado pela razão entre a contagem de Issues fechadas (states: `CLOSED`) e o total de Issues do repositório.

Referências